

How much SPACE do metrics have in GenAI assisted software development?

Samarth Sikand*
Accenture Labs, India
s.sikand@accenture.com

Kanchanjot Kaur Phokela*
Accenture Labs, India
kanchanjot.k.phokela@accenture.com

Vibhu Saujanya Sharma
vibhu.sharma@accenture.com
Accenture Labs, India

Kapil Singi
kapil.singi@accenture.com
Accenture Labs, India

Vikrant Kaulgud
vikrant.kaulgud@accenture.com
Accenture Labs, India

Teresa Tung
teresa.tung@accenture.com
Accenture, United States

Pragya Sharma
pragya.c.sharma@accenture.com
Accenture, India

Adam P. Burden
adam.p.burden@accenture.com
Accenture, United States

ABSTRACT

Large Language Models (LLMs) are revolutionizing the way a developer creates software by replacing code with natural language prompts as primary drivers. While many initial assessments of such LLMs suggest that it helps with developer productivity, other research studies have also pointed out areas in the Software Development Life Cycle (SDLC) and developer experience where such tools fail miserably. Currently, there exist many studies dedicated to evaluation of LLM-based AI-assisted software tools but there lacks a standardization of studies and metrics which may prove to be a hindrance to adoption of metrics and reproducible studies. The primary objective of this survey is to assess the recent user studies and surveys, aimed at evaluating different aspects of developer's experience of using code-based LLMs, and highlight any existing gaps among them. We have leveraged the SPACE framework to enumerate and categorise metrics from studies conducting some form of controlled user experiments. In Generative AI assisted SDLC, the developer's experience should encompass the ability to perform the in-hand task efficiently and effectively, with minimal friction using these LLM tools. Our exploration has led to some critical insights regarding complete absence of user studies in Collaborative aspects of teams, bias towards certain LLM models & metrics and lack of diversity in metrics within productivity dimensions. We also propose some recommendations to the research community which will help bring some conformity in the evaluation of such LLMs.

CCS CONCEPTS

• **Computing methodologies** → **Natural language generation; Artificial Intelligence.**

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ISEC 2024, February 22–24, 2024, Bangalore, India

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1767-3/24/02

<https://doi.org/10.1145/3641399.3641419>

KEYWORDS

Generative AI, Software, SDLC, Metrics, Developer Productivity

ACM Reference Format:

Samarth Sikand, Kanchanjot Kaur Phokela, Vibhu Saujanya Sharma, Kapil Singi, Vikrant Kaulgud, Teresa Tung, Pragya Sharma, and Adam P. Burden. 2024. How much SPACE do metrics have in GenAI assisted software development?. In *17th Innovations in Software Engineering Conference (ISEC 2024)*, February 22–24, 2024, Bangalore, India. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3641399.3641419>

1 INTRODUCTION

Since the release of OpenAI Codex and Github Copilot, there has been an explosion of LLM-based models which can perform software development related tasks like code generation, code summarization etc. at a human level. Many of such tools are now full-fledged and stable commercial products (e.g., Copilot, Tabnine) which are being used by millions of developers every day. Some of the more recent code based models like open-source Code Llama[12] and StarCoder[10] have demonstrated near state-of-the-art performance in problem solving, PaLM 2[2], GPT-4[11] which are general LLMs also perform extremely well on code related tasks. According to a tracker[18], there are at least 50 models and tools trained specifically for software development tasks.

Academia and industry have increased their efforts in evaluating these models on different aspects of productivity, ethics, privacy, security, compliance etc. One of the earliest and important studies in this regard was conducted by GitHub team[19], which collated Copilot telemetry data from developers' IDEs and tried to determine correlation with metrics calculated using a survey based on the SPACE framework. Some works have tried to evaluate the qualitative aspects of developers' experience of using Copilot-like tools by interviewing participants regarding their usage of such tools and extracting valuable insights from them. Some of them have evaluated the code quality aspects of code LLMs via syntactic similarity, susceptibility to vulnerabilities or bugs, and functional correctness. While the studies focus on separate aspects of LLM evaluation and developer experience, there is a lack of standardization or structure to these evaluation studies. As more evaluation studies emerge, it will become crucial that studies do not concentrate only on a few aspects of developer productivity or SDLC processes. As it stands, we

Category	Description
Satisfaction & Well-being	Developer fulfillment and Developer Efficacy
Performance	Outcome of system or process
Activity	Count of actions & activities. Volume of work items
Communication & Collaboration	How teams communicate and work together
Efficiency	Ability to complete work with minimal interruptions

Table 1: Brief description of SPACE framework

believe there 3 potential problems regarding the evaluation studies today:

- Lack of standardization or categorization of studies emerging in evaluating developer productivity and experience
- Unchecked biases of studies towards only certain aspects of developer experience
- Variance in metrics across studies within productivity dimensions

SPACE framework, proposed by Forgsen et. al.[7], is a rational approach to capturing multiple dimensions associated with developer productivity or experience. SPACE framework’s holistic approach, and its popular adoption in research literature for evaluating developer productivity, make it the optimal framework to set our study. In our study, we approach existing literature studies from a SPACE perspective and analyze for any commonalities in approaches, study design and eventually any gaps among them. To this end, our survey only focuses on studies which have conducted controlled user experiments or user surveys in some manner. By narrowing the scope of the studies, our survey tries to ensure that unbiased developer productivity and experience insights are embedded in our analysis.

Our analysis reveals that Collaboration and Communication dimension of SPACE is completely unchecked by current user studies. It highlights a glaring gap in the evaluation of team dynamics and experiences with Generative AI. Furthermore, it was also observed that metrics used within each SPACE dimension are not holistic representation of a particular dimension. Across studies, we observe significant variance in the metrics used to analyze developer productivity and usage patterns with the GenAI tools/models. Some studies[6] have pointed out the fact that even the commonly used metrics have a weak correlation with human judgement. We propose that a categorical framework and standardization of metrics and studies will help the research community to evaluate the GenAI tools in a more equitable and streamlined manner. In the following sections, we detail our approach to selecting the studies, the metrics used and the setups used. In Section 4, a discussion on the analysis of metrics is conducted, followed by the conclusion.

2 SURVEY ANALYSIS

To understand and align the recent developments in the understanding the role of LLM based model on developer experience, we analyze studies based two factors : *Evaluation objectives* and *Study*

design. The *Evaluation objectives* reflect the comprehensive levels of assessment that the study has undertaken to figure out developer experience and at the same time harmonize with the existing literature to a ensure clear and tangible direction of outcomes. The *Study Design* encourages seamless integration of metrics, used to evaluate with GenAI usage, with an existing framework for evaluating developer experience i.e. SPACE.

Moreover, we observed that definition of each SPACE dimension varied significantly across studies. To draw a baseline for further evaluation, it is crucial to bring forth consistency among the definitions. Hence, baseline definitions for dimensions were formalized, based on literature, to categorize the metrics. The intent was that it should be flexible to support diverse products at the same time facilitating outcome oriented criteria to incorporate every changing technology and risk. Fig 1 gives a preview of the high-level steps undertaken for the survey.

2.1 Selection Procedure

For collecting relevant studies, we conducted an automated search exercise followed by pruning studies based on filters like accepted or published. In this survey, it was essential that studies conducted some form of controlled user studies or user surveys regarding the tool or model under focus. The automated search was conducted via popular digital libraries like ACM Digital Library, IEEE Xplore, Science Direct and Research Gate. The query used for searching contained keywords pertaining to Artificial Intelligence (“AI”, “deep learning”, “LLM”, “Transformer”), Evaluation (“user studies”, “productivity”, “experience”), software development (“software”, “code”, “developer”). We also put a filter for studies conducted after 2020, as popular tools like Copilot and Codex were introduced post that year, which can be considered the initial stages of Generative AI era. The search resulted in hundreds of works, we further pruned the list by checking if the studies were conducting controlled user experiments or user surveys. Some blogs from reputed organizations were also included in our study as they cover and enhance our survey. We had shortlisted 15 studies and removed 3 studies (Total: 12) as they were not published and in pre-print. All of the studies have been published in peer-reviewed conferences and journals including ASE, ACL, NIPS, MAPS, CHI, IUI, PROMISE etc.

2.2 Experimental Setup

As mentioned earlier, in our survey we focus on literature about user studies to better understand the mental model of developers. The setup of these studies influences the metrics used to quantify and qualify the evaluation.

Tool/Model selection dictates the outcomes and also draws attention towards any preference for a particular tool/model. This favouritism can be the result of A) Ease of Availability of Tool B) Ease of Use of Tool and C) Developers’ experience while use of tool. *Models*: We collated the unique list of models used the studies as follows: GenLine[9], OpenAI Codex[5, 13, 16, 17], TransCoder[14] and LLM models such as CodeGen350-Multi, CodeGen2B-Multi, Codex-Cushman, Codex-Davinci-1, Codex-Davinci-2 [5, 16] and CoNaLa, tranX-annot, best-tranX, best-tranx-rerank[17].

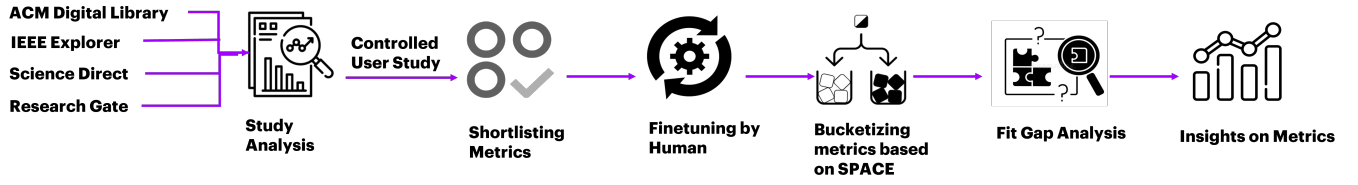


Figure 1: A high-level description of the pipeline of our approach

Secondly, it is important to understand the characteristics of *participants*, like experience, role and previous usage, which can form some underlying assumptions of developer productivity. It was observed that most of the studies have had 20 or more participants. An extensive study from developers with varied experience and profession were used to gauge the results in [3, 19], wherein the users had zero [1] to some experience with the tools. Though developers were experienced, there were certain tasks given to developers in programming languages they had less proficiency [15] to understand the learning curve. While some studies were focused on crowd sourcing, most of them relied on organisation mailing lists to get highly proficient programmers [13, 14]. Fig. 2, depicts the number of participants in the study. As can be observed from the graph, generally studies have a sample size between 30 participants.

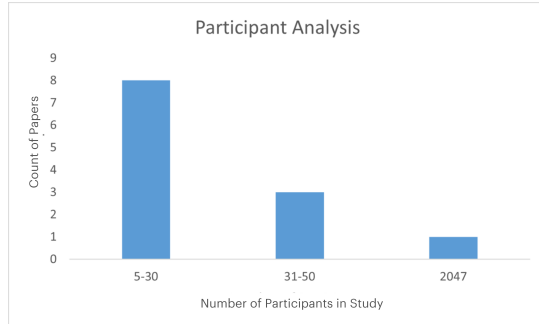


Figure 2: Participant Analysis in the User Study

Finally, the nature of study highlights the *collaboration and pairwise programming effort* in using these tools. While most of the studies were at an individual level, some focused on pair programming with the other end being the LLM model [8]. The studies chose varied time duration for their experiments, wherein some metrics were calculated offline over a period of some days [4, 19] while others were more controlled on a much shorter duration [1, 5, 9, 14, 15]. In some randomized studies, users were given the same set of problems to undertake with and without the assistance of LLM tools [13].

2.3 Study Formation: Dataset and Metrics

To evaluate the efficacy and accuracy of the results provided by the metrics and analyse the implicit and explicit insights, these studies have used various methodologies to support individual claims.

The study setup for developers plays an important role towards the objective of any study. The tasks designed for the surveyed studies can be classified into 2 categories: custom tasks and leveraging existing datasets. The surveyed studies setups included : setup

task [3, 4, 8, 9, 13–15, 19] or IRB approved dataset [1] and HumanEval dataset [5], Leetcode code questions [16] and CoNaLa dataset Heart-Stone dataset [6] to complete the analysis. For quantifying results as metrics, many studies relied heavily on user surveys post experimentation, some studies collected telemetry data and few studies collected log files & Screenshots of developer sessions. [6, 9].

The authors in some studies [3, 4, 6, 9, 14, 15, 19] relied on common, standard metrics as well as proposed a novel metric to differentiate their work. While custom and novel metrics were valid enough to evaluate setup conditions, some studies [1, 5, 8, 13] also tried to normalize their results by evaluating on common known standards.

2.4 SPACE aligned Metrics

As observed in previous sections, we noted that study design corresponded to developer characteristics and nature of evaluation. According to our analysis, the metrics used in shortlisted studies had high variance in terms of context and use i.e., different metrics (custom and common) to evaluate vulnerabilities in code generated. The lack of standardization among metrics serves as an obstacle in the comprehension and comparison of results across studies. In an attempt to normalize these metrics, we leverage SPACE framework [7], and its dimensions definitions, to analyze the metrics compiled and categorize them appropriately. Table 2 highlights the classification of the metrics used in the study based on the categories of SPACE framework. The components of SPACE framework are as follows.

S-Satisfaction and Well-Being: Satisfaction and Well-being of the developer directly and/or indirectly impact the productivity of the developer. In a study, the authors evaluated the code suggestions provided by the LLM tools based on the maintainability and code complexity [1]. While these are software metrics, complex code decreases the overall productivity of the code due to decreased developer efficacy. Similarly, Cognitive load can hamper the overall satisfaction and **Developer efficacy** of a developer. Low fixation count on code tokens/blocks can point to high **Trust** in GenAI outputs. Having high trust could mean one doesn't have to focus on the output to verify it and hence, reducing cognitive load.

P-Performance: Performance of the developer is proportional to the number of positive outcomes, intellect and ability to perform the task well [7]. It is also a good representative of **quality** [1, 8, 9] and adaptability aspects of using a tool. Developer's performance can be linked to GenAI tool's performance, as quality and accuracy of tool's output will influence number of positive outcomes from developer. Performance also includes an aspect of **reliable** outputs or outcomes. In our survey, metrics associated with maintainability of generated outputs can be associated with performance as well.

A-Activity: Activity is the measure of actions or outputs completed in the course of performing work. Although it is not an absolute measure of productivity of developers, it is a very important and crucial part of the evaluation. **Operational Activities** like unit tasks completed or performed have long been the primary metric for many productivity measurements. In our context, the persistence of any GenAI generated output in our final tasks will contribute towards activities performed. Developers may **Handoff** some of the trivial tasks to GenAI tools. The accuracy of the output influences the acceptance of the output[4] by the user, thereby accelerating the time taken to complete the task.

C-Communication and Collaboration: Communication and Collaboration is a measure of the cohesiveness of performing the task as a team. If done effectively, this reduces the overall time taken to perform a task and allows easy handshakes between various tasks.

Metric Name	SPACE category	Sub-category
Halstead Vocabulary [1]	Satisfaction	Developer Efficacy
Coding style adherence [1]	Satisfaction	Style
Halstead Volume [1]	Satisfaction	Developer Efficacy
Halstead Difficulty [1]	Satisfaction	Developer Efficacy
Rank [16]	Satisfaction	Developer Efficacy
Cognitive Load [16]	Satisfaction	Developer Efficacy
Fixation Count [1]	Satisfaction	Trust
First-Fixation Duration [1]	Satisfaction	Trust
Single-Fixation Duration [1]	Satisfaction	Trust
Total Time [1]	Satisfaction	Trust
Uncertainty indicator [14]	Satisfaction	Trust
Attention visualization [14]	Satisfaction	Trust
Severe CWEs/LoC [13]	Performance	Quality
Average CWEs for function [13]	Performance	Quality
Acceptance Rate V2 [9]	Performance	Operational Activity
ndiff [8]	Performance	Developer Efficacy
ROUGE-L [6]	Performance	Quality
METEOR [6]	Performance	Quality
ChrF [6]	Performance	Quality
RUBY [6]	Performance	Quality
CWEs/LineOfCode [13]	Performance	Quality
pass@k [5]	Performance	Quality
CODE-BLEU [6]	Performance	Quality
BLEU [5]	Performance	Quality
Normalized Edit Similarity [5]	Performance	Quality
Code Size [1]	Performance	Quality
Microsoft Visual Studio Maintainability [1]	Performance	Reliability
Acceptance Rate [4, 19]	Activity	Operational Activity
Persistence Rate [4, 19]	Activity	Handoff
Fuzzy persistence rate [19]	Activity	Handoff
Completions Accepted [19]	Activity	Handoff
Contribution speed [4, 19]	Activity	Operational Activity
Acceptance frequency [19]	Activity	Operational Activity
Persistence frequency [19]	Activity	Handoff
Acceptance Rate (2) [9]	Activity	Operational Activity
ndiff [8]	Activity	Operation Activity
Accuracy [16]	Activity	Operation Activity
Shown Rate [19]	Efficiency	Flow
Total Volume [4, 19]	Efficiency	Flow
Loquaciousness [19]	Efficiency	Flow
Eagerness [19]	Efficiency	Flow
Task Completion Times [15]	Efficiency	Flow
Completion Utility [16]	Efficiency	Flow
Highlight Utility [16]	Efficiency	Flow
Time spent in interaction Mode vs (Language Expertise, Prior Copilot Usage, Nature of task) [3]	Efficiency	Design and Coding
Percent time in each prompting strategy [3]	Efficiency	Design and Coding
Percent time spent in each validation strategy [3]	Efficiency	Design and Coding

Table 2: Metrics Compilation in correspondence to SPACE

E-Efficiency and Flow: Efficiency and flow is the measure of completing the task, without any friction from the individual or the system. Hence, the distinct completions and completions[15, 19] per prompt provided by the LLM tool point towards the flow of getting the task done. As prompts have become primary drivers, developers spend a lot of time trying to verify which prompting strategy works best for them. A grounded study[3] has shown that developers have different modes of interacting with such tools. The time taken in aforementioned tasks can determine how efficient

is it to use such tools. It captures the **Flow** of developers and how developers are tackling the design and coding aspects of completing any task.

3 TRENDS AND DISCUSSION

3.1 Biases in Study Design

A basic statistic analysis presents (see Fig.3a) a clear distribution of the overall tool usage across the studies. The majority of the controlled user studies highlight that Github Copilot is the most preferred tool, which is followed by Codex. Few studies experiment with different models within Codex family of models like *code –ushman* and *code –davinci*. It is also established that Github Copilot is essentially powered by Codex models. This implies that majority of studies are dominated by 1 single family of models i.e. Codex. Although it may speak volumes about Copilot’s popularity and its availability as a stable commercial product, but unfortunately it also points to the fact that despite more than 40 code-based LLMs present almost negligible user studies are done using the rest of the LLMs. Most user studies conducted to understand the impact of Generative AI on developers are being formed by less than 2 tools and this might not be a true representation of the entire landscape.

Recommendation: Researchers should attempt to diversify the tool landscape on which user studies are conducted. Conducting user studies across the tools landscape will help reinforce tool’s impact on developers by making it more generalizable.

In Fig.3b, the distribution across Satisfaction, Performance, Activity and Efficiency dimensions is similar which can be viewed as a positive highlight. However one glaring gap can be seen for Collaboration and Communication dimension, as no study has attempted to cover how such tools affect productivity and collaboration as a team. While individual productivity is of utmost importance, one cannot ignore the importance of analyzing team experience and dynamics, as most software organizations would have mid to large sized teams working on a project or product. A whole dimension of insights remain obscure due to absence of studies focusing on GenAI’s impact on interaction/collaboration between individuals with diverse skill sets or roles (e.g., integration, UI/UX).

Recommendation: Researchers must focus on understanding the impact of Gen AI on team dynamics and collaborative aspects. It will help gauge the real value of such tools in an enterprise settings.

3.2 Lack of diverse sub-categories within dimensions

In Fig 3c, one can observe the distribution of metrics along various sub-categories for a specific SPACE dimension. An unbalanced trend emerges from at least 2 dimensions. In the Performance dimension, majority of metrics (e.g., Vulnerabilities, BLEU) are related to Code Quality aspects. While Quality is crucial, it does not cover the much wider spectrum of performance outcomes like maintainability and other reliability characteristics. Within Satisfaction, the metrics which assess coding style (e.g., Coding Style adherence) are considerably lesser than metrics which can gauge Trust (e.g., lower fixation count, low Uncertainty indicator) in such tools. The lack of diverse sub-categories, within metrics, can lead to missing out on some crucial aspects within each SPACE dimension. It can also be observed that there is a lack of uniformity, regarding use of

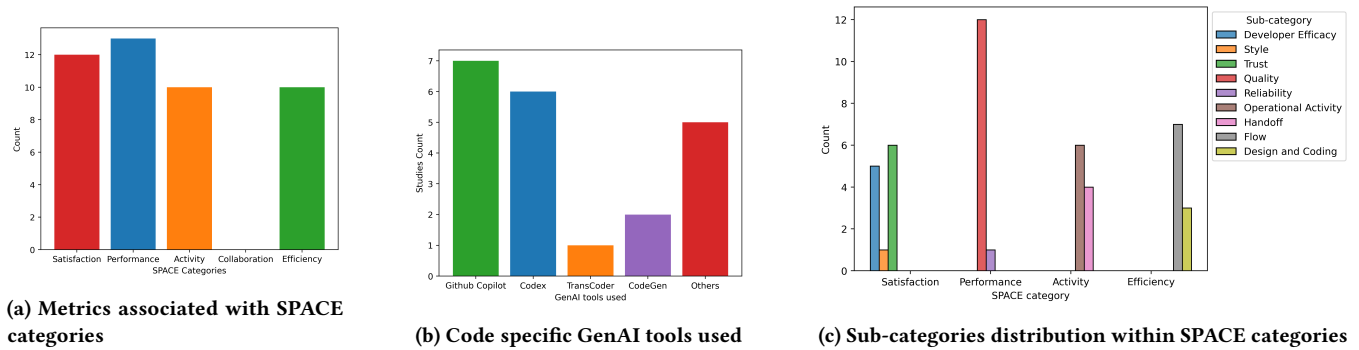


Figure 3: Overall approach to Green AI Quotient

metrics, among studies falling within each dimension. This makes it very challenging to compare different studies or tools equitably.

Recommendation: Increasing focus on bringing diverse metrics within each dimension and standardizing them. Such an evaluation framework can lead to a more holistic evaluation of tools.

Survey Limitations : While we delve into the detailed analysis of metrics, challenges and opportunities mentioned in the studies, there may exist many metrics which might not fall under the purview of SPACE framework. Furthermore, some metrics had some degree of overlap with multiple dimensions but for the purpose of the paper we have kept the categorization mutually exclusive. To assign a metric to a dimension, we investigated and deliberated the level or degree of its overlap with each dimension and assigned it to the dimension with highest degree of overlap.

4 FUTURE WORK AND CONCLUSIONS

In this paper, we have surveyed controlled user/developers studies, associated with Generative AI usage, and put forth key concepts, metrics, and techniques for understanding Developer’s experience. We have aligned and standardized the metrics, used in the studies, based on SPACE framework. For future work, we plan to incorporate a broader and comprehensive framework than SPACE which allows diverse metrics (related to GenAI performance, security etc.) to be captured. It would enable us to align metrics, used in existing and future studies, more accurately and efficiently. Currently we leveraged mutually exclusive categories to make our work precise, but for future framework sophisticated metrics can be mapped across multiple dimensions. The aim of this survey is to promote understanding of the evolution of evaluating LLMs, which is not driven by just core productivity metrics but overall satisfaction of developer and teams.

REFERENCES

- [1] Naser Al Madi. 2022. How readable is model-generated code? examining readability and visual inspection of github copilot. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–5.
- [2] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403* (2023).
- [3] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2022. Grounded copilot: How programmers interact with code-generating models.(2022). *CoRR arXiv 2206* (2022).
- [4] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2023. Taking Flight with Copilot. *Commun. ACM* 66, 6 (2023), 56–62.
- [5] Victor Dibia, Adam Fourney, Gagan Bansal, Forough Poursabzi-Sangdeh, Han Liu, and Saleema Amershi. 2022. Aligning Offline Metrics and Human Judgments of Value of AI-Pair Programmers. *arXiv preprint arXiv:2210.16494* (2022).
- [6] Mikhail Evtikhiev, Egor Bogomolov, Yaroslav Sokolov, and Timofey Bryksin. 2023. Out of the bleu: how should we assess quality of the code generation models? *Journal of Systems and Software* 203 (2023), 111741.
- [7] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of Developer Productivity: There’s more to it than you think. *Queue* 19, 1 (2021), 20–48.
- [8] Saki Imai. 2022. Is github copilot a substitute for human pair-programming? an empirical study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 319–321.
- [9] Ellen Jiang, Edwin Toh, Alejandra Molina, Kristen Olson, Claire Kayacik, Aaron Donsbach, Carrie J Cai, and Michael Terry. 2022. Discovering the syntax and strategies of natural language programming with generative language models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–19.
- [10] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
- [11] OpenAI. 2023. GPT-4 Technical Report. *arXiv:2303.08774* [cs.CL]
- [12] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [13] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. 2022. Security implications of large language model code assistants: A user study. *arXiv preprint arXiv:2208.09727* (2022).
- [14] Jiao Sun, Q Vera Liao, Michael Muller, Mayank Agarwal, Stephanie Houde, Kartik Talamadupula, and Justin D Weisz. 2022. Investigating explainability of generative AI for code through scenario-based design. In *27th International Conference on Intelligent User Interfaces*. 212–228.
- [15] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.
- [16] Helena Vasconcelos, Gagan Bansal, Adam Fourney, Q Vera Liao, and Jennifer Wortman Vaughan. 2022. Generation probabilities are not enough: improving error highlighting for AI code suggestions. In *Virtual Workshop on Human-Centered AI Workshop at NeurIPS (HCAI@NeurIPS’22). Virtual Event, USA*. 1–4.
- [17] Frank F Xu, Bogdan Vasilescu, and Graham Neubig. 2022. In-side code generation from natural language: Promise and challenges. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 2 (2022), 1–47.
- [18] Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Wang Yongji, and Jian-Guang Lou. 2023. Large Language Models Meet NL2Code: A Survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 7443–7464. <https://doi.org/10.18653/v1/2023.acl-long.411>
- [19] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.